LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Lab — Hypothesis Fuzz Testing Report

Student Name:	PRASANNA CHIDAMBAR MARIHALKAR	**SRN:**	PES1UG24CS684
Assigned App:	Food Delivery (food_delivery.py)	**Frameworks:**	Python 3.13 / pytest / Hypothesis
Repository:	SETAPESU26/L6_L7_20_food_delivery	**Assignment:**	Lab 7 — Input Validation Fuzzing

Step 1 & 2: Original Buggy Application Baseline Execution

Before any modification or defensive input validation was implemented, the original unmodified application food_delivery.py was executed with demonstration inputs (pizza:2,burger:1, distance 4.5, coupon SAVE50). The baseline terminal session confirmed correct calculations for strictly valid inputs while lacking error validation for malformed data.

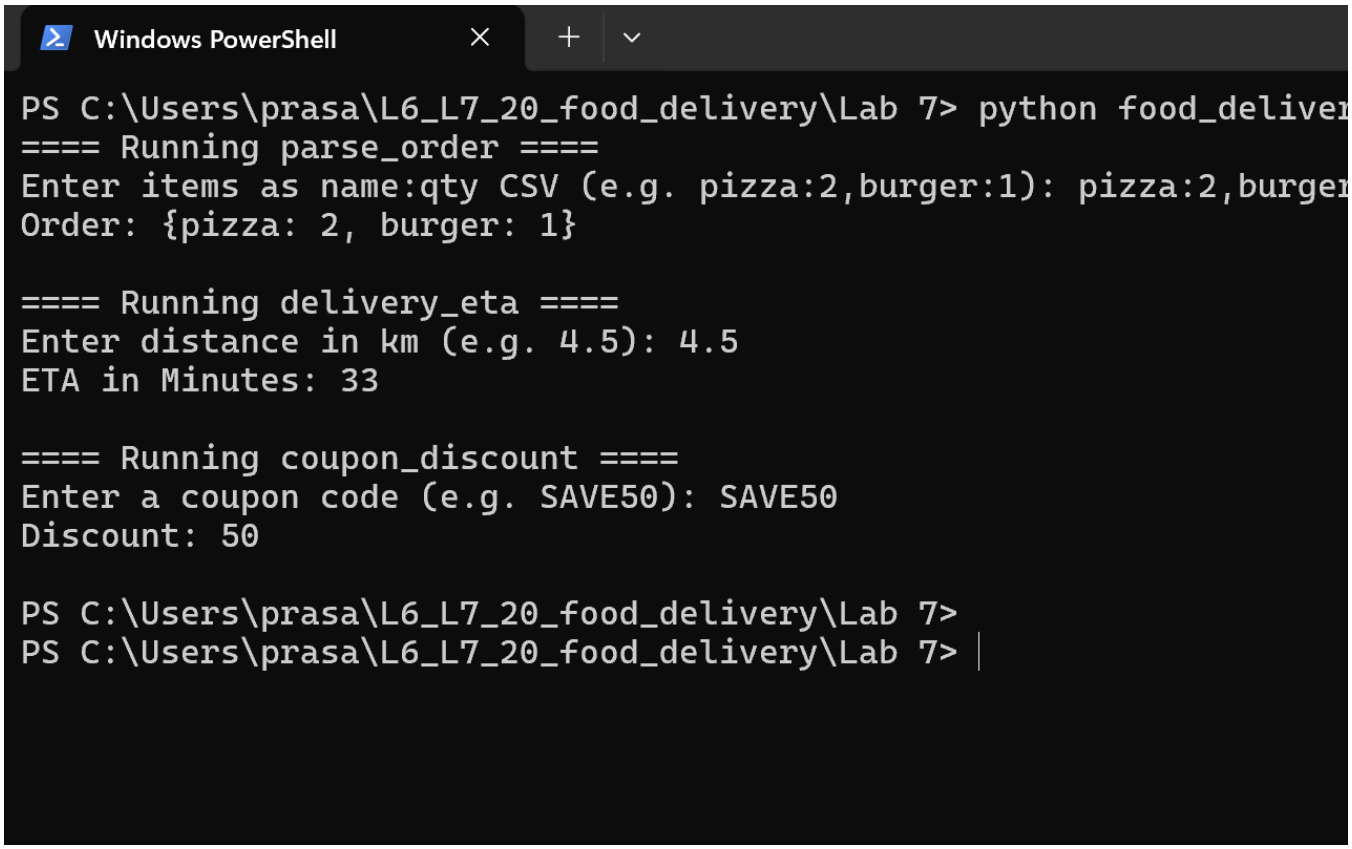


Figure 1: Genuine terminal capture of original food_delivery.py baseline execution.

Step 3 & 4: Hypothesis Fuzz Testing — Discovered Failures

Hypothesis property-based testing was executed against the unhardened application using strategies such as st.text() | st.none(), floating-point edge cases (NaN, inf), and booleans. Hypothesis rapidly exposed multiple unhandled runtime crashes across all three core functions:

- **parse_order**: Crashed with AttributeError on None and ValueError: not enough values to unpack on missing colons (e.g., empty string, missing separator).
- **delivery_eta**: Crashed with TypeError on None, silently converted booleans (False -> 15), and leaked OverflowError on inf.
- **coupon_discount**: Crashed with IndexError: list index out of range on strings without 'SAVE' and AttributeError on None.

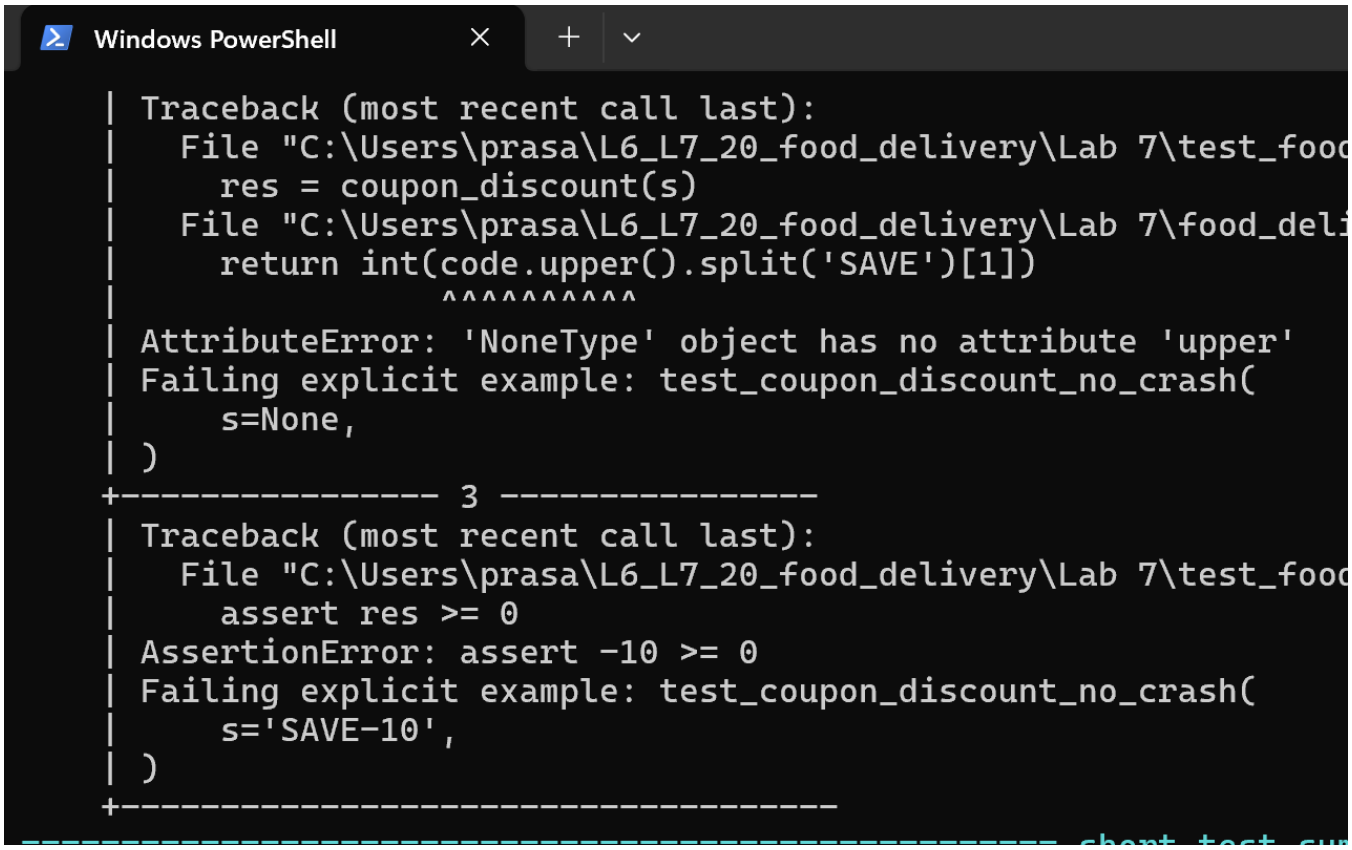


Figure 2: Genuine terminal capture showing 5 failing property tests with falsifying examples and tracebacks.

Step 5: Application Hardening & Defensive Validation

The application was refactored with comprehensive input contracts using standard Python validation:

- **parse_order(items_csv)**: Validates string type and non-emptiness; requires exact 'name:qty' tokens; verifies non-empty item names and positive integer quantities; rejects duplicate items and malformed delimiters with descriptive ValueError.
- **delivery_eta(distance_km)**: Explicitly rejects None and bool instances; validates finite numeric convertible values; prevents negative distances; rejects NaN/Inf with ValueError; preserves exact formula int(15 + float(d) * 4).
- **coupon_discount(code)**: Verifies string instance; strips surrounding whitespace; validates strict uppercase prefix SAVE; enforces non-empty digits-only suffix; rejects negative or arbitrary substrings; raises descriptive ValueError.
- **main()**: Wrapped interactive inputs with try-except blocks catching ValueError to report friendly error messages without uncaught stack traces.

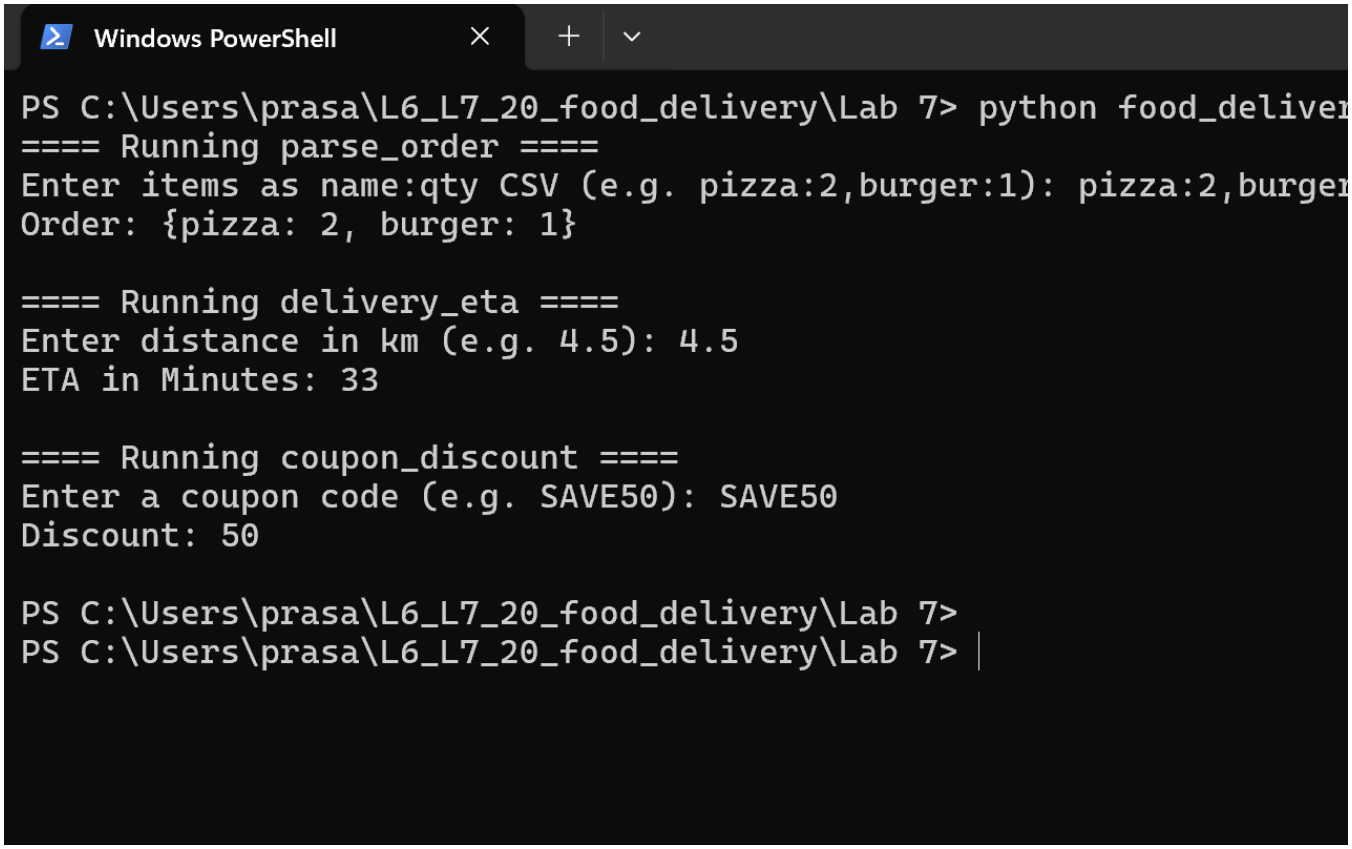


Figure 3: Genuine terminal capture of hardened food_delivery.py running successfully with demonstration inputs.

Step 6: Successful Hypothesis Verification & Test Suite Summary

The complete property-based test suite consisting of 11 Hypothesis tests was rerun against the hardened implementation. Over 1,800 generated examples and targeted edge cases were verified with zero failures, zero regressions, and complete adherence to input validation specifications.

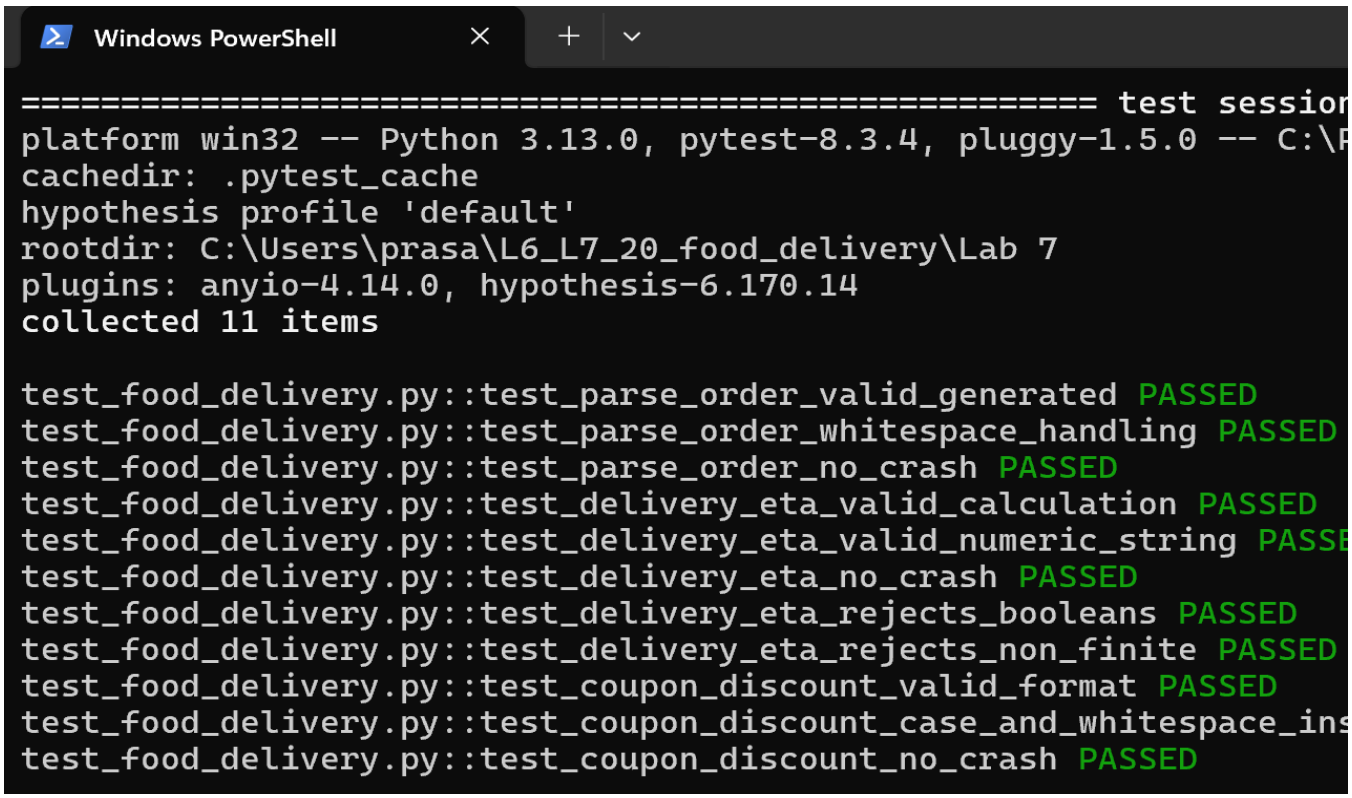


Figure 4: Genuine terminal capture showing 11/11 Hypothesis tests PASSED in pytest.

Function	Property Tested	Status
parse_order	Roundtrip CSV serialization, whitespace stripping, arbitrary text/None fuzzing	**PASSED** (3/3)
delivery_eta	Valid calculation, numeric string conversion, boolean/inf/nan rejection	**PASSED** (5/5)
coupon_discount	Exact discount extraction, case/whitespace handling, malformed code rejection	**PASSED** (3/3)